STATEMENT OF WORK

AWS Cloud Infrastructure Setup

Project: SimplyiCard

Prepared For	SimplyiCard
Prepared By	Intern Cloud Team
Region	AWS us-east-1 (N. Virginia)
Budget Goal	Kept as low as possible, under $150
Version	v1.0
Date	August 20, 2026

Table of Contents

1. Background
2. Goals
3. Scope
4. Architecture
5. Networking
6. Compute (Containers)
7. Load Balancer
8. Database
9. CI/CD Pipeline
10. Monitoring & Logs
11. Deliverables
12. Technical Requirements
13. Security Requirements
14. Tasks
15. Timeline
16. Acceptance Criteria
17. Budget
18. Sign-off

1. Background

SimplyiCard is a containerized web application that requires a secure, reliable, scalable, and cost-efficient cloud environment for deployment and operation. This project focuses on designing and implementing the required AWS infrastructure to support application hosting, networking, database connectivity, security, monitoring, and application delivery.

The proposed infrastructure will use AWS services including Amazon VPC, Amazon ECS with EC2, Amazon ECR, Application Load Balancer, Amazon RDS, AWS Certificate Manager, IAM, Amazon CloudWatch, Amazon S3, and AWS CloudTrail. The architecture will follow standard AWS security and networking practices, with appropriate separation of application and database resources.

Cost optimization will be considered throughout the design by selecting appropriately sized resources, avoiding unnecessary infrastructure, and using suitable AWS pricing options while maintaining the required security, performance, and functionality.

This SOW defines the project scope, technical requirements, deliverables, security considerations, cost optimization approach, and estimated budget for the SimplyiCard AWS infrastructure.

2. Goals

The main goal is to build a secure, reliable, and low-cost AWS setup to host the SimplyiCard app. The setup should:

- Design and implement a secure, reliable, and cost-optimized AWS infrastructure for hosting the SimplyiCard containerized web application.
- Separate the application and database layers using appropriate public and private subnet configurations, security groups, and controlled network access to protect sensitive resources.
- Deploy the infrastructure across two Availability Zones to improve availability and minimize the impact of a failure in a single Availability Zone.
- Implement an automated CI/CD deployment process to build, manage, and deploy application updates with minimal manual intervention.
- Use appropriate AWS services and resource configurations to optimize performance while avoiding unnecessary infrastructure and operational costs.
- Maintain the overall AWS infrastructure cost below **$150 per month**, based on the expected workload and applicable AWS pricing benefits.
- Implement monitoring, logging, and auditing capabilities to provide visibility into infrastructure health, application performance, and AWS resource activity.
- Establish a scalable infrastructure foundation that can accommodate future application growth with minimal architectural changes.

3. Scope

This project covers the design, configuration, and deployment of the AWS infrastructure required to host the SimplyiCard containerized web application. It includes the configuration of the AWS networking

environment, container hosting, application load balancing, database infrastructure, security controls, automated CI/CD deployment, monitoring, logging, and cost optimization.

The project includes setting up the required VPC, subnets, routing, security groups, ECS environment, ECR repository, Application Load Balancer, RDS database, IAM permissions, HTTPS configuration, and monitoring services. The CI/CD pipeline will automate application build and deployment activities to reduce manual intervention and ensure consistent releases.

The implementation will be performed within the **AWS us-east-1 region** and will focus on the infrastructure requirements defined in this SOW. Ongoing day-to-day infrastructure administration, application feature development, major application code changes, and deployments or infrastructure outside the defined AWS region are considered **out of scope**.

4. Architecture

The proposed SimplyiCard architecture provides a secure, reliable, and cost-optimized AWS environment for deploying the containerized application. The infrastructure is deployed within a dedicated Amazon VPC in the us-east-1 region, with application resources distributed across two Availability Zones to improve availability and resilience.

User traffic enters through an Application Load Balancer (ALB)**,** which distributes requests to the SimplyiCard application running in containers on Amazon ECS with EC2 launch type across the two Availability Zones. Container images are securely stored in Amazon ECR and deployed to ECS through the CI/CD pipeline.

The application and database layers are logically isolated to protect sensitive data. Amazon RDS is hosted in private subnets and is not directly accessible from the internet. Database access is restricted to the application layer through appropriate security groups and network controls**.**

Application code is maintained in GitHub**,** and code updates flow through the automated CI/CD pipeline, where the application is built, containerized, stored in ECR, and deployed to the ECS environment with minimal manual intervention.

The architecture also incorporates IAM for access control, AWS Certificate Manager (ACM) for HTTPS/TLS, Amazon CloudWatch for monitoring and logging, and AWS CloudTrail for auditing AWS activity. The architecture diagram below illustrates the complete traffic flow, application and database connectivity, AWS networking components, and automated deployment process.

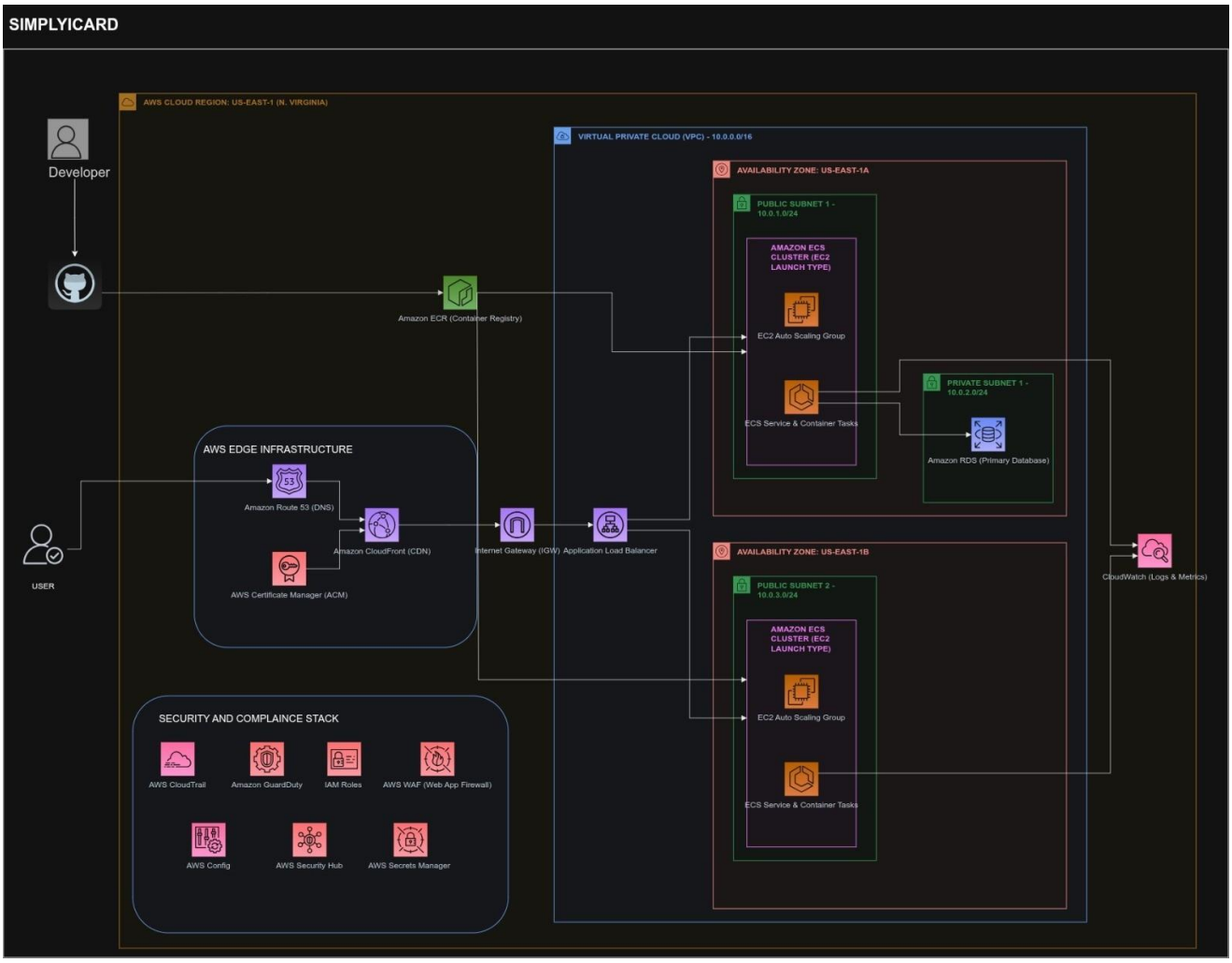


Figure 1 — SimplyiCard AWS setup

Key Ideas

- **Layered Security:** Network access is controlled through the Application Load Balancer and security groups, ensuring that only authorized traffic can reach the application. The database is isolated from public access and can only be accessed by the application layer.
- **High Availability:** Application resources are distributed across two Availability Zones to improve resilience and reduce the impact of a failure affecting a single zone.
- **Automated Deployments:** Application code changes are managed through a centralized CI/CD pipeline that automates the build, container image management, and deployment process, minimizing manual intervention and deployment errors.
- **Cost Optimization:** AWS resources are selected and sized according to the expected workload, with eligible cost-efficient and Free Tier options considered where applicable. Unnecessary resources and resource consumption are minimized to keep operational costs under control.

5. Networking

- **Amazon VPC:** A dedicated Virtual Private Cloud will be configured in the **us-east-1 region** to provide an isolated and controlled network environment for the SimplyiCard infrastructure.

- **Availability Zones:** The infrastructure will utilize **two Availability Zones (us-east-1a and us-east-1b)** to improve application availability and resilience against a failure affecting a single zone.
- **Public Subnets:** Two public subnets will be configured across the Availability Zones to host the **Application Load Balancer and ECS application resources** that require controlled internet connectivity.
- **Private Subnets:** Two private subnets will be configured for the **Amazon RDS database layer**, preventing direct access from the public internet and providing network-level isolation.
- **Internet Gateway:** An Internet Gateway will provide internet connectivity for resources that require external communication, with routing controlled through appropriate route tables and security controls.
- **Security Groups:** Security groups will be configured as stateful network controls to restrict inbound and outbound traffic based on the specific communication requirements between the load balancer, application, and database layers.

6. Compute (Containers)

- **Amazon ECS:** Amazon Elastic Container Service (ECS) will be used to deploy, manage, and run the SimplyiCard application as containerized workloads.
- **EC2 Launch Type:** ECS will use the EC2 launch type, providing control over the underlying compute instances while supporting cost-efficient container hosting.
- **ECS Cluster:** A dedicated ECS cluster will be configured to provide the compute environment for running the application containers.
- **ECS Service:** An ECS service will maintain the required number of application tasks and support continuous application availability.
- **ECS Tasks:** Application containers will run as ECS tasks using the defined task definition, including the container image, CPU, memory, networking, and port configuration.
- **Auto Scaling:** EC2 Auto Scaling can be configured to adjust compute capacity based on application requirements, helping maintain availability while avoiding unnecessary resource provisioning.
- **Amazon ECR:** Container images generated through the deployment process will be stored in Amazon Elastic Container Registry (ECR) and pulled by ECS during application deployment

7. Load Balancer

- **Application Load Balancer (ALB):** An Application Load Balancer will serve as the primary entry point for incoming application traffic and distribute requests to the SimplyiCard application running on Amazon ECS.
- **Target Group:** A target group will be configured to route traffic from the ALB to the appropriate ECS service and application tasks.
- **Health Checks:** ALB health checks will continuously verify the availability of the application targets and route traffic only to healthy targets.
- **Multi-AZ Distribution:** The ALB will operate across the configured Availability Zones to improve application availability and provide resilience against a single-zone failure.
- **HTTPS:** HTTPS/TLS termination will be configured using AWS Certificate Manager (ACM) to secure communication between users and the application.
- **Security Controls:** The ALB security group will restrict inbound traffic to the required ports, such as HTTPS (443), while application security groups will allow traffic only from the ALB.

8. Database

- **Amazon RDS:** Amazon Relational Database Service (RDS) will be used to host the SimplyiCard application database, providing a managed and reliable database environment.
- **Private Subnets:** The database will be deployed within **private subnets** across the configured Availability Zones and will not be directly accessible from the public internet.
- **Application-Only Access:** Database connectivity will be restricted to the ECS application layer through appropriate security group rules, allowing only the required database traffic.
- **Single-AZ Configuration:** For the development/staging environment, a Single-AZ RDS configuration will be considered to reduce infrastructure costs while meeting the project's current requirements.
- **Database Security:** Access to the database will be controlled through network security groups and appropriate AWS access policies to protect application data.
- **Backup and Monitoring:** RDS backup and monitoring capabilities will be configured as required to support data protection, availability, and operational visibility.

9. CI/CD Pipeline

- **GitHub Actions:** Automates the CI/CD workflow by triggering the build and deployment process when application code is pushed or changes are merged into the repository.
- **Container Build:** GitHub Actions will build the updated SimplyiCard application into a Docker container image and validate the build as part of the deployment workflow.
- **Amazon ECR:** Amazon Elastic Container Registry (ECR) will securely store and manage the built container images used by the ECS environment.

- **Automated ECS Deployment:** After a successful image build and push to ECR, GitHub Actions will deploy the updated image to the Amazon ECS service, reducing manual deployment activities.
- **Application Availability:** The deployment process will be configured to minimize application downtime during updates and maintain service availability as far as supported by the ECS deployment configuration.
- **Secrets Management:** Sensitive credentials, passwords, access keys, and other deployment secrets will be stored securely using **GitHub Secrets** and will not be hard-coded or committed to the application source code.
- **Version-Controlled Deployment:** GitHub will maintain the application source code and deployment workflow configuration, providing traceability of code changes and supporting consistent application releases.

10. Monitoring & Logs

- **Amazon CloudWatch:** CloudWatch will monitor the health and performance of AWS resources such as ECS, EC2, ALB, and RDS. It will collect metrics, application logs, and infrastructure logs, with alarms configured for important conditions such as high resource utilization, unhealthy targets, or service failures.
- **AWS CloudTrail:** CloudTrail will record AWS API activity and account-level events, providing an audit trail of actions performed on AWS resources for security monitoring, troubleshooting, and operational accountability.
- **Amazon S3:** S3 will provide durable object storage for application files, build artifacts, exported data, and database backup files where required. Appropriate access controls, encryption, and lifecycle policies will be applied to protect data and manage storage costs.
- **AWS Config:** AWS Config will be used to monitor and record the configuration of supported AWS resources. It will help track configuration changes, identify resources that do not comply with defined configuration requirements, and support security and governance activities.
- **Centralized Monitoring and Governance:** These services together will provide visibility into resource performance, application logs, configuration changes, and AWS account activity, helping the team maintain a secure, monitored, and well-managed environment.

11. Deliverables

No.	What's Included	Qty
1	A private network (VPC) split into public and private areas, across two zones	1
2	Internet Gateway and routing setup	1
3	ECS cluster with auto-scaling servers	1
4	The SimplyiCard app running on ECS	1
5	Load balancer with HTTPS set up	1
6	RDS database (db.t3.micro) with subnet group	1
7	Security groups for the load balancer, app, and database	3
8	IAM roles with only the access each part needs	As needed

9	AWS WAF (firewall) attached to the load balancer	1
10	AWS CloudTrail activity log	1
11	Amazon GuardDuty threat detection	1
12	AWS Config for tracking changes	1
13	AWS Security Hub turned on	1
14	Amazon ECR repository	1
15	GitHub Actions pipeline (build, push, deploy)	1
16	CloudWatch logs, metrics, and alerts	As needed

12. Technical Requirements

The setup should meet these requirements:

- The infrastructure shall be deployed in the AWS us-east-1 region using the defined Availability Zones for application resilience.
- A dedicated Amazon VPC shall be configured with public and private subnets, route tables, Internet Gateway, and appropriate network controls.
- The application shall be deployed as Docker containers on Amazon ECS using the EC2 launch type, with appropriate ECS task definitions and service configuration.
- Amazon ECR shall be used as the container image repository for storing and managing application images.
- An Application Load Balancer (ALB) shall provide external application access, traffic distribution, target health checks, and HTTPS/TLS termination.
- Amazon RDS shall host the application database within private subnets and shall not allow direct public internet access.
- Security groups and IAM policies shall be configured according to the principle of least privilege and minimum required network access.
- GitHub Actions shall be used to implement CI/CD automation for building container images, pushing images to ECR, and deploying application updates to ECS.
- AWS Certificate Manager (ACM) shall provide the TLS certificate required for secure HTTPS communication.
- Amazon CloudWatch shall provide infrastructure and application monitoring, log collection, metrics, and alarms.
- AWS CloudTrail and AWS Config shall provide AWS activity auditing and resource configuration tracking.
- AWS WAF, Amazon GuardDuty, and AWS Security Hub shall be configured as applicable to provide web application protection, threat detection, and security visibility.
- Amazon S3 shall be used where required for application artifacts, backup files, or other approved object-storage requirements.

- Sensitive credentials, passwords, API keys, and database connection details shall not be hard-coded in application source code and shall be securely managed through approved secrets-management mechanisms.
- The infrastructure shall be designed with **cost optimization** as a key requirement, using appropriately sized resources and avoiding unnecessary resource consumption while maintaining required functionality and security.
- The deployed environment shall undergo functional, connectivity, security, CI/CD, monitoring, and cost validation before project completion.

13. Security Requirements

Network and Access Security

- **IAM and Least Privilege:** AWS IAM roles and policies will follow the principle of least privilege, providing users and AWS services only the permissions required for their intended operations.
- **Network Security:** Public and private subnets will separate internet-facing resources from backend resources. Security groups will restrict traffic to only the required ports and authorized sources.
- **Load Balancer Access:** The Application Load Balancer will accept required HTTP/HTTPS traffic on ports **80 and 443** from external users.
- **Application Access:** ECS application containers will not be directly accessible from the internet and will accept traffic only from the Application Load Balancer.
- **Database Protection:** Amazon RDS will be deployed without direct public internet access and will accept database connections only from the authorized application layer.
- **Network Isolation:** Application and database resources will remain logically separated to reduce unauthorized access and limit the impact of potential security incidents.

Security Protection and Data Security

- **AWS WAF:** AWS WAF will protect the public-facing application against common web attacks and unwanted HTTP/HTTPS requests.
- **Amazon GuardDuty:** GuardDuty will provide continuous threat detection to identify potentially malicious or unauthorized activity within the AWS environment.
- **Security Monitoring:** AWS CloudTrail, AWS Config, and AWS Security Hub will provide auditing, configuration monitoring, security findings, and centralized security visibility.
- **Encryption:** Public application traffic will be protected using **HTTPS/TLS**, with encryption at rest enabled for supported AWS services where required.
- **Secrets Management:** Passwords, database credentials, API keys, and other sensitive information will not be hard-coded or stored in source code. Secrets will be securely managed and provided to applications during deployment.
- **Security Review:** Security alerts, configuration changes, and identified findings will be monitored and reviewed to support timely detection and remediation of potential security issues.

14. Tasks

- **Planning and Design:** Finalize the AWS architecture, deployment region, Availability Zones, required services, security requirements, and cost estimates before implementation.

- **Network Configuration:** Create and configure the Amazon VPC, public and private subnets, Internet Gateway, route tables, and required network connectivity.
- **Security Group Configuration:** Configure dedicated security groups for the Application Load Balancer, ECS application layer, and RDS database with restricted inbound and outbound traffic.
- **Compute Configuration:** Configure the ECS cluster, EC2 Auto Scaling Group, ECS capacity, task definitions, and ECS service required to run the application containers.
- **Load Balancer Configuration:** Configure the Application Load Balancer, target groups, HTTPS/TLS certificate, listeners, routing rules, and health checks.
- **Database Configuration:** Deploy and configure Amazon RDS within private subnets with appropriate database settings, security controls, and application connectivity.
- **CI/CD Implementation:** Configure GitHub Actions, Docker image builds, Amazon ECR integration, and automated deployment of application updates to ECS.
- **Security Service Configuration:** Enable and configure AWS WAF, AWS CloudTrail, Amazon GuardDuty, AWS Config, and AWS Security Hub as required within the project scope.
- **Monitoring and Logging:** Configure Amazon CloudWatch for metrics, logs, alarms, and operational monitoring, along with Amazon S3 storage where required.
- **Testing and Validation:** Validate HTTPS access, load balancer routing, ECS container health, database isolation, security controls, CI/CD deployment, monitoring, and overall application functionality.
- **Cost Validation:** Review resource utilization and estimated AWS charges before and after deployment to ensure the infrastructure remains within the approved project budget.

15. Timeline

The timeline depends on the project schedule. The goal is to have everything set up and tested within one academic term, or as agreed.

16. Acceptance Criteria

The project will be considered successfully completed when the following criteria have been met:

- The SimplyiCard application is accessible through HTTPS, and the Application Load Balancer successfully routes traffic to healthy ECS application tasks across the configured Availability Zones.
- The Amazon RDS database is not publicly accessible and accepts connections only from the authorized application layer through the configured security controls.
- A test code change pushed to GitHub successfully triggers the CI/CD pipeline, builds and pushes a new container image to Amazon ECR, and deploys the updated application to ECS with minimal or no service interruption.
- Required security services, including AWS WAF, AWS CloudTrail, Amazon GuardDuty, AWS Config, and AWS Security Hub, are enabled and operational as defined within the project scope.
- Amazon CloudWatch successfully collects application and infrastructure logs and metrics, with required monitoring alarms configured and operational.

- The implemented AWS resources and estimated usage are validated against the approved budget, with projected spending maintained within the defined cost target.
- Network connectivity, application functionality, database access restrictions, CI/CD deployment, security controls, monitoring, and logging are validated through appropriate testing before project closure.

17. Budget

This project is built by an intern team, so there's no labor cost. We're aiming to keep AWS spending as low as possible, well under $150, mostly by sticking to Free Tier resources. Here's the estimated monthly breakdown:

Item	Notes	Est. Cost / mo
EC2 (app servers)	Free Tier	$0
RDS (database)	Free Tier	$0
Load balancer	Not free — main cost	$16 – $18 / mo
HTTPS certificate (ACM)	Free	$0
ECR (image storage)	Free Tier	$0 – $1
S3 (backups & files)	Free Tier	$0 – $1
CloudWatch (logs & alerts)	Free Tier	$0 – $2
AWS WAF (firewall)	Small usage fee	$5 – $8 / mo
CloudTrail	Free	$0
GuardDuty	Free trial, then small fee	$0 – $3
AWS Config	Small usage fee	$2 – $4
Security Hub	Free trial, then small fee	$0 – $3
Data transfer	Free Tier	$0 – $2
Labor (intern team)	Unpaid	$0
Estimated Total		**$23 – $39 / mo**

How We Keep Costs Down

- **Free Tier first:** we use EC2, RDS, and free HTTPS certificates wherever the Free Tier covers them.
- **Load balancer is the main cost:** it's the one piece that isn't free, so we budget for it directly.
- **Right-sized security tools:** GuardDuty and Security Hub run on their free trial periods where possible, and Config only tracks what's needed for this project.
- **No wasted spend:** we keep the minimum number of servers running and shut things down when not actively needed.
- **Regular checks:** we check actual AWS costs against the budget before and after launch.

18. Sign-off

By signing below, both sides agree to the plan, deliverables, and budget described in this document.

SimplyiCard Representative	Intern Team Lead
Name & Date	Name & Date